# NETMIRAGE: AUTONOMOUS CYBER-RANGE WITH ADAPTIVE DECEPTION ROUTING

## COMPREHENSIVE ARCHITECTURE GUIDE, CODE WALKTHROUGH & DEFENSE PLAYBOOK

**Course:** BACSE203 P — Computer Networks Lab (Innovative Lab Assessment (ELA 1.0)) | **Institution:** Vellore Institute of Technology (VIT), Vellore

**Faculty Guide:** Dr. Sudha M, Professor, School of Computer Science and Engineering

**Team:** Abhisumat Panda (25BYB0202), Atharva Parmar (25BYB0180), Manas Patel (25BYB0190), Varun Agarwal (25BYB0186)

## CHAPTER 1: THE BIG PICTURE — WHAT IS NETMIRAGE & WHY DOES IT EXIST?

*Modern cybersecurity education faces an unprecedented dilemma: generative AI models and autonomous agents can synthesize sophisticated offensive reconnaissance and lateral movement scripts in seconds. However, unleashing an autonomous AI agent onto physical lab networks or open cloud environments creates catastrophic risks of accidental denial-of-service, unintended IP leakage, self-propagating worms, and host system breakout. NetMirage was conceived, architected, and built as an academic defensive cyber-range that solves this crisis completely. It allows security researchers and students to observe, analyze, and manipulate autonomous AI attack behavior safely in real time without any possibility of host or internet harm.*

### 1.1: The Core Problem: Why Studying Autonomous AI Attackers is Dangerous

**WHAT IT IS (IN SIMPLE ENGLISH):**

Imagine giving an eager learner a real flamethrower inside a crowded wooden house to study how fire spreads. That is exactly what happens when you give an AI attacker raw terminal access on a real network: a single hallucination, infinite retry loop, or unexpected subnet scan can crash hospital servers, corrupt databases, or leak private credentials to the public internet.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

Large Language Models (LLMs) and autonomous reinforcement-learning planners exhibit non-deterministic decision pathways. When provided with an unrestricted command-line interface (e.g., bash/sh) or raw network sockets, AI agents suffer from three fundamental architectural hazards:

1. Hallucinatory Amplification: The model may generate invalid CIDR ranges or broadcast floods (e.g., ping sweeps against 255.255.255.255 or public subnets like 8.8.8.8) that trigger upstream ISP abuse flags.
2. Arbitrary Code Execution & Host Escape: Allowing an agent to execute arbitrary shell scripts permits model prompt injections where an adversarial decoy payload instructs the AI to execute 'rm -rf /' or spawn reverse shells back to the developer's host OS.
3. Cascading Resource Exhaustion: Without hardware and concurrency boundaries, an autonomous agent can spawn thousands of parallel socket connections, causing TCP SYN floods that exhaust ephemeral ports and crash the testing host.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`app_build/backend/core/safety_policy.py (SafetyPolicyEngine.evaluate_action) & app_build/docker/docker-compose.lab.yml`

### 1.2: The NetMirage Solution: Disposable Lab, Action Broker, & Dynamic Deception

**WHAT IT IS (IN SIMPLE ENGLISH):**

NetMirage acts like a high-tech stunt arena surrounded by blast-proof glass. The AI attacker thinks it is breaking into a real corporate enterprise, but every computer it touches is a harmless toy decoy (honeypot). As soon as the AI tries to steal passwords or scan for secrets, our system automatically shifts the fake maze to steer the attacker deeper into containment while we watch every move on a live radar screen.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

NetMirage implements a multi-tier cyber-range architecture governed by five synchronized defense layers:

1. Disposable Zero-Egress Lab: Five isolated Docker bridge networks (10.0.10.0/24 to 10.0.50.0/24) with 'internal: true' ensuring mathematical impossibility of internet routing.
2. Allowlisted Action Broker: A mediating proxy between the AI Planner and the network. The AI cannot run shell commands; it can only submit structured JSON parameter requests for one of 7 allowlisted actions.
3. Passive Protocol Analyzers: Telemetry taps that inspect Layer 2 ARP frames, Layer 3 IP headers, Layer 4 TCP states, and Layer 7 application transactions without altering host configurations.
4. MITRE ATT&CK; Classification Engine: Real-time mapping of observed wire behavior to industry-standard adversary techniques (e.g., T1046 Network Service Discovery, T1110 Brute Force, T1071.004 DNS Exfiltration) with confidence scoring.
5. Dynamic Deception Policy Engine: An adaptive state machine that recalculates the attacker's threat score and reroutes subsequent adversary interactions away from perimeter services into specialized honeynet zones (Cowrie SSH, Web decoys, DNS/SMTP decoys, or a quarantine sinkhole).

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`app_build/backend/orchestration/lab_runner.py (LabRunner._run_loop) & app_build/backend/intel/deception_policy.py`

### 1.3: The Non-Negotiable Boundary: Zero Shell, Zero Host, Zero Internet

**WHAT IT IS (IN SIMPLE ENGLISH):**

We built an unbreakable vault door: no matter how smart or broken the AI gets, it cannot touch the real operating system, it cannot access the internet, and it cannot execute any shell commands. It is like an actor reading from a menu of 7 safe buttons—it cannot invent an 8th button.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

NetMirage enforces three non-negotiable security boundaries that are verified at runtime by automated test suites:

1. Zero Arbitrary Shell Execution: The codebase completely forbids 'subprocess.Popen(shell=True)', 'os.system()', or direct bash piping. Only 7 strict functions in ActionBroker can be executed:
- discover_lab_services (Port sweep simulation)
- resolve_dns (Internal domain lookup)
- connect_tcp (Three-way handshake test)
- request_http (Decoy web path GET/POST)
- attempt_ssh_to_lure (Cowrie credential authentication)
- send_icmp_probe (Decoy echo probe)
- inspect_session_output (Decoy session terminal harvest)

2. Zero Host Breakout: Docker Compose strictly mandates 'cap_drop: ALL', 'security_opt: [no-new-privileges:true]', 'read_only: true', and forbids 'network_mode: host' or 'privileged: true'.

3. Zero Egress: Docker networks declare 'internal: true', which drops default gateway routing to external interfaces.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`app_build/backend/core/safety_policy.py (ALLOWLISTED_ACTIONS & validate_container_security)`

### 1.4: Academic Lab Alignment: VIT BACSE203 P Computer Networks Lab

**WHAT IT IS (IN SIMPLE ENGLISH):**

This project is not just a theoretical report—it is a working, physical implementation of every single experiment taught in the VIT Computer Networks curriculum, from subnet calculation and ARP resolution to TCP handshakes and HTTP protocol exchanges.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

NetMirage maps directly to the official syllabus modules of BACSE203 P:
- Module 1 (Layer 2 / Data Link): Ethernet framing, ARP table inspection, and ARP cache poisoning detection.
- Module 2 (Layer 3 / Network): IPv4 Classless Inter-Domain Routing (CIDR /24 subnetting), IP header TTL, ICMP echo probes.
- Module 3 (Layer 4 / Transport): Socket programming, TCP 3-way handshake analysis, half-open SYN scan detection, port registries.
- Module 4 (Layer 7 / Application): Domain Name System (DNS A records), HTTP/1.1 REST endpoints, SSH-2.0 protocol banners, SMTP ESMTP mail exchanges, and WebSockets (RFC 6455).
- Module 5 & 6 (Network Security & Deception): Defensive deception routing, MITRE ATT&CK; matrix correlation, and honeypot design.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`docs/ARCHITECTURE.md, docs/VIVA_PREP_GUIDE.md, and production_artifacts/viva_data.py`

# CHAPTER 2: END-TO-END LIFECYCLE OF A SINGLE RUN (STEP-BY-STEP STORY)

*To truly understand how NetMirage operates, we follow the complete chronological life of a single adversarial emulation run. From the millisecond the human operator presses 'START RUN' to the instant the global kill switch terminates all active processes, every event flows through a deterministic, fail-closed pipeline.*

## STAGE 1: OPERATOR INITIATES RUN FROM DASHBOARD

**WHAT IT IS (IN SIMPLE ENGLISH):**

The user clicks the glowing green 'START RUN' button on the web console. This sends a quick 'wake-up' message to the backend server saying: 'Create a brand new experiment run right now.'

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

The React frontend invokes 'createRun()' in 'app_build/frontend/src/services/api.ts', triggering an HTTP POST request to 'http://localhost:8000/api/runs' with a JSON payload specifying the scenario ('full_adversary_emulation'). The server generates a unique run identifier 'run_YYYYMMDD_HHMMSS_xxxx' and persists a record in SQLite with status 'pending'.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`app_build/frontend/src/App.tsx (handleStartRun) -> app_build/backend/routes.py (create_run)`

## STAGE 2: CONTROL API SPAWNS LAB RUNNER & ESTABLISHES WEBSOCKET STREAM

**WHAT IT IS (IN SIMPLE ENGLISH):**

The server starts the background emulation engine and opens a live, two-way walkie-talkie channel (WebSocket) with the browser so the dashboard can display live updates without refreshing.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

FastAPI's 'create_run' handler passes the new 'run_id' to 'lab_runner.start_run(run_id)', which creates an asynchronous task via 'asyncio.create_task(self._run_loop(run_id))'. Simultaneously, the React frontend connects to 'ws://localhost:8000/api/runs/{run_id}/events'. The Event Bus allocates a dedicated 'asyncio.Queue' for this connection, emits an initial system envelope, and begins streaming events in real time.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`app_build/backend/orchestration/lab_runner.py (start_run) & app_build/backend/event_bus.py (subscribe)`

## STAGE 3: AI ATTACKER EXECUTES 3-LAYER PLANNING LOOP

**WHAT IT IS (IN SIMPLE ENGLISH):**

The AI attacker looks around at what it has learned so far (Observation), thinks about what to do next (Planner), and writes a safe, polite request to test a specific port (Safe Action Request).

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

Inside '_run_loop', the engine executes the 3-layer adversary loop:

Layer 1: 'planner.build_observation_summary()' synthesizes prior actions, discovered nodes, and open ports into an ObservationSummary dataclass.

Layer 2: 'planner.plan_next_action()' selects the next tactical objective from the deterministic cyber-range progression state machine.

Layer 3: Generates a 'SafeActionRequest' dataclass containing action_type, target_ip, target_port, parameters, and rationale.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`app_build/backend/core/planner.py (AIPlanner.build_observation_summary & plan_next_action)`

## STAGE 4: SAFETY POLICY ENGINE EVALUATES GUARDRAILS (FAIL-CLOSED)

**WHAT IT IS (IN SIMPLE ENGLISH):**

Before anything touches the network, a strict security guard checks 7 rules: Is this action on the approved list? Is the destination IP inside our lab? Is the port registered? Is the emergency kill switch active? If anything fails, the action is instantly blocked.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

'safety_policy.evaluate_action()' performs seven sequential checks:

1. Action Allowlist: Must be in ALLOWLISTED_ACTIONS.
2. Kill Switch: Verifies 'run.status == running'; rejects if paused or terminated.
3. Duration Cap: Rejects if run duration exceeds 300 seconds.
4. CIDR Verification: Target IP must fall strictly within allowed lab networks (10.0.10.0/24 to 10.0.50.0/24).
5. Port Registry: Target port must belong to ALLOWED_LAB_PORTS (22, 53, 80, 8080, 25, 587, 2222).
6. Rate Limiting: Max 60 actions per minute sliding window.
7. Concurrency Lock: Max 5 concurrent active operations. Fails closed (allowed=False, status='denied') on any error.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`app_build/backend/core/safety_policy.py (SafetyPolicyEngine.evaluate_action)`

### STAGE 5: ACTION BROKER EXECUTES ALLOWLISTED NETWORK OPERATION

**WHAT IT IS (IN SIMPLE ENGLISH):**

Once approved, the Action Broker performs the exact requested action (like sending a SYN packet or an HTTP request), measures how many milliseconds it took, and bundles the result into clean JSON data.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

'action_broker.execute_action()' routes the operation to its dedicated handler (e.g., '_exec_discover_lab_services', '_exec_request_http', '_exec_attempt_ssh_to_lure'). It measures execution latency via 'time.perf_counter()', releases the concurrency counter in a 'finally' block, and returns an Action Broker Result dictionary: {action, target, port, status: 'completed', latency_ms: int, output: dict}.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`app_build/backend/core/action_broker.py (ActionBroker.execute_action)`

### STAGE 6: HONEYPOT DECOY INGESTION & UNIVERSAL CREDENTIAL REDACTION

**WHAT IT IS (IN SIMPLE ENGLISH):**

If the attacker tried to log into our decoy SSH server with a fake password like 'SuperSecretPassword123!', our honeypot recorder immediately scrubs that password and replaces it with '[REDACTED]' so no secret leaks.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

When the action targets a honeypot (SSH, Web, DNS, or SMTP), the parameters pass through 'honeypot_normalizer'. The 'redact_credentials()' engine recursively scans dictionaries, lists, strings, and query parameters using regex to replace passwords, auth tokens, and API keys with '[REDACTED]'. A sanitized HoneypotSession record is saved to SQLite.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`app_build/backend/intel/honeypots.py (redact_credentials & HoneypotNormalizer)`

### STAGE 7: PROTOCOL TELEMETRY EXTRACTION BY PROTOCOL ANALYZER

**WHAT IT IS (IN SIMPLE ENGLISH):**

Our passive network wire-tap notices the traffic: 'Aha! The attacker just sent a rapid burst of TCP SYN packets across the edge gateway!' It records this as a behavioral observation without touching the server.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

'protocol_analyzer.analyze_action_event()' converts the executed action into low-level network observations. For a service sweep, it generates an observation for 'port_discovery' with raw payload flags 'SYN' and incomplete handshake metrics. For SSH, it records 'ssh_credential_probing'; for DNS queries containing anomalies, it logs 'dns_anomaly'.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`app_build/backend/intel/analyzer.py (ProtocolAnalyzer.analyze_action_event)`

### STAGE 8: MITRE ATT&CK; CLASSIFICATION & CONFIDENCE SCORING

**WHAT IT IS (IN SIMPLE ENGLISH):**

The system looks up the hacker playbook (MITRE ATT&CK;) and says: 'This port sweep is officially Technique T1046 (Network Service Discovery) with 90% confidence!' It also bumps up the total danger meter (Threat Score).

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

'mitre_mapper.classify_observation()' evaluates the observation against 'MAPPING_RULES'. It maps 'port_discovery' to MITRE technique 'T1046' (Network Service Discovery, confidence 0.90, profile: 'reconnaissance'). It assigns technique T1110 (Brute Force) to SSH probing, and T1071.004 to DNS anomalies. The cumulative threat score increments towards 100.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`app_build/backend/intel/mitre_mapper.py (MitreAttackMapper.classify_observation)`

### STAGE 9: DYNAMIC DECEPTION POLICY RECALCULATES TARGET ROUTING

**WHAT IT IS (IN SIMPLE ENGLISH):**

Now the defense system strikes back intelligently: 'Because the attacker is doing reconnaissance, don't let them touch real servers—change the route so their next request lands straight into our Cowrie SSH honeypot!'

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

'deception_policy.evaluate_policy()' consumes the threat profile and confidence. Reconnaissance triggers dynamic diversion to 'ssh-lure' (10.0.20.10) or 'web-lure' (10.0.30.10). Credential brute forcing diverts the adversary to 'services-lure' (10.0.40.0/24). Network tampering triggers diversion to the 'quarantine' sinkhole (10.0.50.99). The decision is emitted over WebSocket.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`app_build/backend/intel/deception_policy.py (DeceptionPolicyEngine.evaluate_policy)`

## STAGE 10: OPERATOR HITS TERMINATE (KILL SWITCH & TEARDOWN)

**WHAT IT IS (IN SIMPLE ENGLISH):**

At any time, the user can click 'TERMINATE'. Instantly, all background loops freeze, the containers stop, and the run is safely sealed as completed.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

Clicking 'TERMINATE' fires POST '/api/runs/{run_id}/terminate'. FastAPI sets 'run.status = terminated' in SQLite and calls 'lab_runner.terminate_run(run_id)', which invokes 'asyncio.Task.cancel()'. The loop catches 'asyncio.CancelledError', emits a final system event, releases concurrency locks, and cleanly tears down container network allocations.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`app_build/backend/routes.py (terminate_run) & app_build/backend/orchestration/lab_runner.py (terminate_run)`

# CHAPTER 3: COMPUTER NETWORKS CONCEPTS USED IN NETMIRAGE

*NetMirage is designed as an end-to-end pedagogical demonstration of classical computer networking theory applied to modern defensive cybersecurity. This chapter dissects the exact networking fundamentals, mathematics, packet structures, and RFC protocols running inside the system.*

## 3.1: VLAN Segmentation & IPv4 Subnetting Math (The 5 Lab Subnets)

**WHAT IT IS (IN SIMPLE ENGLISH):**

Subnetting is like building separate soundproof rooms in an office building. Rather than putting everyone on one loud, chaotic floor, we assign distinct IP ranges so computers in Room A cannot talk to Room B without going through a controlled gateway.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

NetMirage establishes five distinct Classless Inter-Domain Routing (CIDR) /24 subnets:

- Edge Subnet: 10.0.10.0/24 (Netmask 255.255.255.0, Gateway 10.0.10.1, Broadcast 10.0.10.255, Usable: 10.0.10.2 - 10.0.10.254). Hosts Edge Gateway (10.0.10.42) and Attacker Ingress (10.0.10.15).
- SSH Decoy Subnet: 10.0.20.0/24 (Gateway 10.0.20.1). Hosts Cowrie SSH Lure (10.0.20.10).
- Web Decoy Subnet: 10.0.30.0/24 (Gateway 10.0.30.1). Hosts Web Decoy Portal (10.0.30.10).
- Core Services Subnet: 10.0.40.0/24 (Gateway 10.0.40.1). Hosts DNS Lure (10.0.40.10) and SMTP Lure (10.0.40.20).
- Monitor & Quarantine Subnet: 10.0.50.0/24 (Gateway 10.0.50.1). Hosts Telemetry Sensor (10.0.50.10) and Quarantine Sinkhole (10.0.50.99).

Docker enforces network isolation via Linux network bridges created with 'internal: true'. This parameter instructs the Docker daemon to configure iptables with a DROP rule on any packet destined for external network interfaces (eth0/wlan0), preventing internet egress.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`app_build/docker/docker-compose.lab.yml (networks section) & app_build/backend/core/safety_policy.py (is_ip_in_lab)`

## 3.2: Layer 2 (Data Link Layer): ARP Framing & MITRE T1557 Detection

**WHAT IT IS (IN SIMPLE ENGLISH):**

Before two computers can talk using IP addresses, they must find each other's physical hardware address (MAC address) using ARP: 'Who has IP 10.0.10.42? Tell 10.0.10.15!' If a hacker lies about who they are, that is ARP poisoning.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

Address Resolution Protocol (ARP, RFC 826) maps 32-bit IPv4 addresses to 48-bit MAC addresses within a broadcast domain. In an Ethernet frame, the EtherType field is set to 0x0806 for ARP and 0x0800 for IPv4. NetMirage's Protocol Analyzer monitors Gratuitous ARP broadcasts and rapid MAC-to-IP mapping changes. If an attacker attempts an Adversary-in-the-Middle (AiTM) ARP spoofing attack, the Protocol Analyzer detects the conflicting MAC advertisement and flags MITRE Technique T1557 with a confidence score of 0.92, triggering dynamic rerouting into the quarantine sinkhole.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`app_build/backend/intel/analyzer.py (arp_spoof_candidate) & app_build/backend/intel/mitre_mapper.py (T1557)`

## 3.3: Layer 3 (Network Layer): IPv4 Headers, TTL & ICMP Anomaly Detection (T1095)

**WHAT IT IS (IN SIMPLE ENGLISH):**

Every packet travelling across our subnets has an envelope containing source and destination IPs, plus a 'Time To Live' (TTL) counter that prevents packets from looping forever. We also send ICMP 'ping' messages to see if decoy servers are awake.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

An IPv4 packet header consists of 20 bytes minimum: Version (4 bits), IHL (4 bits), Type of Service (8 bits), Total Length (16 bits), Identification, Flags (DF, MF), Fragment Offset, Time to Live (TTL, 8 bits), Protocol (8 bits, e.g., 1 for ICMP, 6 for TCP, 17 for UDP), Header Checksum, and 32-bit Source/Destination IP addresses.

NetMirage tests ICMP (Internet Control Message Protocol, RFC 792) via Type 8 (Echo Request) and Type 0 (Echo Reply). The Protocol Analyzer evaluates ICMP payload entropy. Attackers frequently hide exfiltrated data inside the optional payload field of ICMP ping packets. When anomalous payload entropy (>4.0) is detected, the system maps this to MITRE Technique T1095 (Non-Application Layer Protocol).

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`app_build/backend/core/action_broker.py (_exec_send_icmp_probe) & app_build/backend/intel/analyzer.py (icmp_payload_anomaly)`

## 3.4: Layer 4 (Transport Layer): TCP 3-Way Handshake vs Half-Open SYN Scans (T1046)

**WHAT IT IS (IN SIMPLE ENGLISH):**

To establish a reliable connection, TCP does a 3-step handshake: 'Hello!' (SYN), 'Hello back, ready!' (SYN-ACK), and 'Great, let us talk!' (ACK). Attackers cheat by sending only the first 'Hello' to see if a door is unlocked, then hanging up immediately with a 'Reset' (RST) to avoid being logged.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

Transmission Control Protocol (TCP, RFC 793) is connection-oriented and utilizes 16-bit source/destination ports and 32-bit Sequence/Acknowledgment numbers. A normal connection executes: Client sends SYN -> Server responds SYN-ACK -> Client sends ACK.

In stealth reconnaissance (Nmap SYN scan), the attacker sends SYN; if the server replies SYN-ACK (port open), the attacker immediately replies with RST, terminating the half-open connection before application logs trigger. NetMirage's Protocol Analyzer inspects incomplete handshake ratios. When sequential SYN sweeps across lab ports (22, 53, 80, 8080) are observed, it generates a 'port_discovery' observation that maps to MITRE Technique T1046 (Network Service Discovery, confidence 0.90).

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`app_build/backend/intel/analyzer.py (port_discovery) & app_build/backend/intel/mitre_mapper.py (T1046)`

### 3.5: Layer 7 (Application Layer): HTTP, SSH-2.0, DNS, SMTP, REST & WebSockets

**WHAT IT IS (IN SIMPLE ENGLISH):**

At the top layer, real programs communicate: browsers send HTTP web requests, admins log into SSH terminals, computers look up names via DNS, and our frontend talks to our backend over super-fast WebSockets that stay open 24/7.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

NetMirage exercises five major Layer 7 protocols:

1. HTTP/1.1 (RFC 2616): Request methods (GET, POST), URIs, and status codes (200 OK). Decoy web lures inspect path probing (/admin/login) mapping to MITRE T1190.
2. SSH-2.0 (RFC 4253): Medium-interaction Cowrie lure emulates OpenSSH 8.9p1. Performs Diffie-Hellman / Curve25519 key exchange, harvests adversary authentication attempts, and sanitizes passwords into [REDACTED] (MITRE T1110 / T1021.004).
3. DNS (RFC 1035): Port 53 UDP/TCP resolution of 'A' records (e.g., 'ssh.corp.internal' -> 10.0.20.10). Anomaly detector identifies covert DNS tunneling queries (MITRE T1071.004).
4. SMTP (RFC 5321): Port 25 decoy mail server emulating ESMTP banners and recipient probing.
5. WebSockets (RFC 6455): Upgraded from HTTP via 'Upgrade: websocket' headers, establishing persistent full-duplex bi-directional frame streaming between FastAPI backend and React frontend.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`app_build/backend/intel/honeypots.py & app_build/backend/event_bus.py & app_build/frontend/src/services/websocket.ts`

# CHAPTER 4: COMPLETE CODEBASE TOUR (FILE-BY-FILE WALKTHROUGH)

*This chapter provides an exhaustive technical tour of every file, class, function, and Docker container inside '/app_build/'. It explains the architectural responsibility of each file and how they interlock to form the NetMirage cyber-range platform.*

## app_build/backend/main.py — Application Entry Point & FastAPI Lifespan

**WHAT IT IS (IN SIMPLE ENGLISH):**

The front door of our backend program. It starts the server, configures CORS security, and mounts all the web routes.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

Initializes the FastAPI application instance ('app = FastAPI(title="NetMirage Defensive Cyber-Range")'). Configures CORSMiddleware to allow cross-origin requests from the Vite frontend (http://localhost:5173). Defines the async 'lifespan' context manager that calls 'init_db()' on startup to ensure SQLite tables exist, and mounts the primary APIRouter from 'routes.py'.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

**`Exports: init_db(), lifespan(), app.include_router(api_router)`**

## app_build/backend/routes.py — RESTful Control Endpoints & WebSocket Event Stream

**WHAT IT IS (IN SIMPLE ENGLISH):**

The traffic controller. It handles buttons clicked on the web UI (like Start, Pause, Terminate) and powers the live WebSocket cable.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

Defines 11 RESTful endpoints and 1 WebSocket handler:

- GET /api/health: Returns operational state, active run count, and zero-egress status.
- GET /api/topology: Returns the 5 VLAN segments, active honeypot lures, and highlighted deception paths.
- POST /api/runs: Creates a new run, commits to database, and triggers 'lab_runner.start_run()'.
- POST /api/runs/{id}/pause, resume, terminate: Manages run lifecycle and kill switches.
- GET /api/runs/{id}/actions, detections, policies: Fetches historical forensics.
- GET /api/sessions/{session_id}: Fetches honeypot sessions with credential redaction applied.
- WS /api/runs/{id}/events: Subscribes client to EventBus queue, emits init envelope, and streams live EventEnvelopes.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

**`Exports: get_health(), get_topology(), create_run(), pause_run(), terminate_run(), websocket_events()`**

## app_build/backend/models.py & db.py — SQLAlchemy 2.0 ORM & The 6 Immutable Tables

**WHAT IT IS (IN SIMPLE ENGLISH):**

The permanent memory vault. It creates a local SQLite database with 6 organized tables that store every event forever.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

'db.py' configures the SQLite engine ('sqlite:///./netmirage.db') with 'check_same_thread=False' and establishes 'SessionLocal'. 'models.py' declares the 6 immutable tables using SQLAlchemy 2.0 ORM with ForeignKeys and cascading deletions:

1. Run (runs): id, status, threat_score, active_profile, next_deception_zone, created_at, started_at, ended_at.
2. AgentAction (agent_actions): id, run_id, timestamp, action_type, target_ip, target_port, status, latency_ms, output.
3. Observation (observations): id, run_id, timestamp, source, protocol, behavior_type, raw_payload, details.
4. Detection (detections): id, run_id, timestamp, source_ip, technique_id, technique_name, confidence, evidence.
5. HoneypotSession (honeypot_sessions): id, run_id, lure_type, session_id, source_ip, commands_or_requests, metadata_json.
6. PolicyDecision (policy_decisions): id, run_id, timestamp, threat_profile, confidence, decision, target_zone.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

**`Exports: Run, AgentAction, Observation, Detection, HoneypotSession, PolicyDecision, init_db(), get_db()`**

## app_build/backend/event_bus.py — Universal Event Bus & Database Event Sinks

**WHAT IT IS (IN SIMPLE ENGLISH):**

The central loudspeaker. Whenever an action, detection, or route change happens, Event Bus broadcasts it to the browser and saves it in the database.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

Maintains an in-memory subscription map 'Dict[str, Set[asyncio.Queue]]' and history cache 'Dict[str, List[EventEnvelope]]' guarded by an 'asyncio.Lock()'. The 'publish()' method distributes incoming envelopes to all subscriber queues and invokes '_persist_event()', which maps the envelope kind ('system', 'action', 'observation', 'detection', 'honeypot', 'policy') directly into SQLite table rows within a transactional commit/rollback block.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`Exports: EventBus.subscribe(), EventBus.unsubscribe(), EventBus.publish(), EventBus.emit(), EventBus._persist_event()`

### app_build/backend/core/planner.py — 3-Layer Adversary Emulation Planner

**WHAT IT IS (IN SIMPLE ENGLISH):**

The simulated brain of the hacker. It looks at what it already discovered and chooses the next tactical move from a list of safe steps.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

Implements a 3-layer architecture for safe adversary emulation:

1. Layer 1 (Observation Summary): 'build_observation_summary()' parses previous actions and observations into an ObservationSummary dataclass.
2. Layer 2 (Planner): 'plan_next_action()' advances through DETERMINISTIC_PROGRESSION (7-stage cyber-range progression: discovery -> DNS -> Web -> SSH brute force -> Session harvest -> ICMP probe -> TCP verify).
3. Layer 3 (Safe Action Request): Returns a 'SafeActionRequest' dataclass with strict typing.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`Exports: ObservationSummary, SafeActionRequest, AIPlanner.build_observation_summary(), AIPlanner.plan_next_action()`

### app_build/backend/core/safety_policy.py — Zero-Trust Fail-Closed Safety Policy Engine

**WHAT IT IS (IN SIMPLE ENGLISH):**

The strict bouncer. It checks every requested action against CIDR boundaries, registered ports, rate limits, and kill switches before allowing it.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

'SafetyPolicyEngine' implements strict fail-closed security. Checks include: membership in 'ALLOWLISTED_ACTIONS', 'run.status == running', duration <= 300s, IP in 'allowed_networks' (via ipaddress.ip_network containment check), port in 'allowed_ports', sliding window rate limit <= 60 actions/min, and active concurrency <= 5. Returns a 'PolicyResult(allowed=True/False, status="allowed"/"denied", reason=str)'. Any check failure immediately halts execution.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`Exports: SafetyPolicyEngine.evaluate_action(), is_ip_in_lab(), is_port_allowed(), check_rate_limit(), validate_container_security()`

### app_build/backend/core/action_broker.py — Mediated Action Broker (Zero Arbitrary Shell Execution)

**WHAT IT IS (IN SIMPLE ENGLISH):**

The hands and feet of the system. It executes the 7 approved actions, measures latency, and returns structured JSON.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

Translates abstract requests into concrete operations without raw shell calls. Contains seven isolated methods:

- '_exec_discover_lab_services': Returns subnet-tailored service inventories.
- '_exec_resolve_dns': Resolves internal corporate hostnames via mock authoritative DNS table.
- '_exec_connect_tcp': Simulates TCP 3-way handshake completion.
- '_exec_request_http': Emulates HTTP GET/POST to decoy endpoints (/admin/login).
- '_exec_attempt_ssh_to_lure': Simulates SSH-2.0 authentication handshake with banner exchange.
- '_exec_send_icmp_probe': Simulates ICMP Echo Request/Reply with TTL verification.
- '_exec_inspect_session_output': Harvests decoy session keystroke logs.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`Exports: ActionBroker.execute_action(), _exec_discover_lab_services(), _exec_resolve_dns(), _exec_attempt_ssh_to_lure()`

### app_build/backend/orchestration/lab_runner.py — Lab Orchestration & Run Loop Lifecycle

**WHAT IT IS (IN SIMPLE ENGLISH):**

The master conductor. It ties together the planner, safety engine, broker, analyzers, and deception router in a smooth loop.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

Manages active asyncio background tasks for running emulations. The '_run_loop(run_id)' method coordinates the 10-step lifecycle:
Checks run status in DB -> Generates observation summary -> Plans action -> Evaluates safety policy -> Executes via ActionBroker -> Normalizes honeypot interactions -> Runs ProtocolAnalyzer -> Classifies via MitreAttackMapper -> Evaluates DeceptionPolicyEngine -> Emits envelopes over EventBus -> Paces execution with asyncio.sleep(). Handles cancellation and exceptions cleanly.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

**Exports: LabRunner.start_run(), pause_run(), resume_run(), terminate_run(), _run_loop()**

### app_build/backend/intel/analyzer.py — Passive Protocol Telemetry Analyzer

**WHAT IT IS (IN SIMPLE ENGLISH):**

The wire-tap. It inspects packet traffic to spot port scans, SSH brute force attacks, and hidden data channels.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

'ProtocolAnalyzer' derives passive network observations from action events. Generates structured observation payloads for:
- 'port_discovery': Sequential SYN sweeps across lab subnets.
- 'tcp_syn_pattern': Established TCP handshakes with RTT latency.
- 'web_path_probing': HTTP URI probing of administrative paths.
- 'ssh_credential_probing': SSH cipher negotiations and auth attempts.
- 'dns_anomaly': Anomaly detection on domain length or 'exfil' substrings.
- 'icmp_payload_anomaly': High entropy or oversized echo payloads.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

**Exports: ProtocolAnalyzer.analyze_action_event(), ProtocolAnalyzer.analyze_scapy_packet()**

### app_build/backend/intel/mitre_mapper.py — MITRE ATT&CK; Classification Engine

**WHAT IT IS (IN SIMPLE ENGLISH):**

The threat intelligence index. It tags network traffic with official MITRE IDs (like T1046) and calculates confidence scores.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

'MitreAttackMapper' matches behavioral observations against 'MAPPING_RULES':
- port_discovery -> T1046 (Network Service Discovery, 0.90)
- web_path_probing -> T1190 (Exploit Public-Facing App, 0.85)
- ssh_credential_probing -> T1110 (Brute Force, 0.95)
- dns_anomaly -> T1071.004 (Application Layer: DNS, 0.88)
- arp_spoof_candidate -> T1557 (Adversary-in-the-Middle, 0.92)
- icmp_payload_anomaly -> T1095 (Non-App Layer Protocol, 0.80)

Returns a 'MitreDetection' dataclass containing technique_id, technique_name, confidence, threat_profile, and evidence.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

**Exports: MitreDetection, MitreAttackMapper.classify_observation()**

### app_build/backend/intel/deception_policy.py — Dynamic Deception Policy Engine

**WHAT IT IS (IN SIMPLE ENGLISH):**

The active defense strategist. It decides whether to route the attacker to an SSH honeypot, a fake website, or a quarantine box.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

'DeceptionPolicyEngine.evaluate_policy()' consumes threat profiles and confidence scores:
- 'reconnaissance' -> Reroutes adversary to 'ssh-lure' (10.0.20.10) or 'web-lure' (10.0.30.10).
- 'credential_probing' -> Diverts adversary deeper into 'services-lure' (10.0.40.0/24).
- 'network_manipulation' -> Immediately restricts adversary to 'quarantine' sinkhole (10.0.50.99).
- 'covert_channel' -> Mirrors traffic to 'dns-lure' tap (10.0.40.10).

Returns a 'DeceptionDecision' dataclass that updates 'run.next_deception_zone'.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

**Exports: DeceptionDecision, DeceptionPolicyEngine.evaluate_policy()**

### app_build/backend/intel/honeypots.py — Honeypot Decoy Normalizers & Universal Credential Redaction

**WHAT IT IS (IN SIMPLE ENGLISH):**

The sanitizer. It cleans stolen passwords and sensitive data into '[REDACTED]' before saving to the database or showing on screen.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**
Contains 'redact_credentials(obj)': A recursive scrubbing engine that traverses dicts, lists, and strings. Redacts dictionary keys matching SENSITIVE_KEYS ('password', 'secret', 'token', 'key', 'auth_token', 'api_key'). Uses regex to scrub key-value strings (password=xxx -> password=[REDACTED]), CLI user flags (-u user:pass -> -u user:[REDACTED]), and Basic Auth headers. 'HoneypotNormalizer' defines standard normalizers for Cowrie SSH, Web lures, DNS taps, and SMTP decoys.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

**`Exports: redact_credentials(), HoneypotNormalizer.normalize_cowrie_event(), normalize_web_event(), normalize_dns_event()`**

### app_build/docker/docker-compose.lab.yml — Hardened Zero-Egress Docker Cyber-Range Lab

**WHAT IT IS (IN SIMPLE ENGLISH):**

The physical laboratory. Spawns 5 isolated virtual network bridges and 5 decoy microservice containers with zero internet access.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**
Defines 5 bridge networks with 'internal: true' (edge_net 10.0.10.0/24, ssh_net 10.0.20.0/24, web_net 10.0.30.0/24, services_net 10.0.40.0/24, monitor_net 10.0.50.0/24). Defines 5 microservices:
- edge-gateway (10.0.10.42)
- cowrie-ssh-lure (10.0.20.10)
- web-lure (10.0.30.10)
- dns-smtp-lure (10.0.40.10)
- telemetry-collector (10.0.50.10)
Every container enforces: 'security_opt: [no-new-privileges:true]', 'cap_drop: [ALL]', 'read_only: true', 'tmpfs: [/tmp:rw,noexec,nosuid]', and strict CPU/memory limits (0.5 CPUs, 256MB RAM).

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

**`Exports: Networks: edge_net, ssh_net, web_net, services_net, monitor_net. Services: edge-gateway, cowrie-ssh, web-lure, dns-smtp, collector`**

### app_build/frontend/src/App.tsx — React 18 Operator Console Root & State Machine

**WHAT IT IS (IN SIMPLE ENGLISH):**

The commander's screen. Ties together the top control bar, live network map, terminal feed, detection matrix, and honeypot viewer.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**
Central state coordinator written in React 18 with TypeScript. Manages state for: runs, currentRun, topology, actions, detections, policyDecisions, sessions, activeTargetIp, selectedFilterNode, and wsStatus. Mounts 'RunWebSocketClient' on run change, dynamically handles incoming EventEnvelopes, and distributes state to the 5 specialized child components.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

**`Exports: App(), handleIncomingEvent(), handleStartRun(), handlePauseRun(), handleResumeRun(), handleTerminateRun()`**

### app_build/frontend/src/components/NetworkTopology.tsx — Interactive VLAN Topology & Dynamic Rerouting Canvas

**WHAT IT IS (IN SIMPLE ENGLISH):**

The visual network map. Shows all 5 subnets as clean cards with glowing animated lines indicating where traffic is being redirected.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**
Renders an interactive SVG/HTML5 schematic displaying the 5 VLAN segments in high-contrast Ivory (#FAF5E9) and Charcoal (#111827). Nodes render with IP, role, and status badges. Glowing SVG polylines animate traffic paths from attacker (10.0.10.15) to active targets, turning Tangerine Yellow (#FFCC00) when traffic is diverted into deception honeypots.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

**`Exports: NetworkTopology(segments, activeTargetIp, highlightedPaths, selectedFilterNode, onSelectNode)`**

### app_build/frontend/src/components/TopControlBar.tsx — Operator Control Header & Threat Telemetry Badges

**WHAT IT IS (IN SIMPLE ENGLISH):**

The top dashboard dashboard. Lets the operator start, pause, resume, or terminate runs, and shows the live Threat Score meter.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**
Houses the run selection dropdown, execution buttons (Start Run, Pause, Resume, Terminate Kill Switch), Threat Score progress meter (0 to 100), active MITRE threat profile tag, current deception target zone tag, and live WebSocket heartbeat indicator.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`Exports: TopControlBar(runs, currentRun, onSelectRun, onStartRun, onPauseRun, onResumeRun, onTerminateRun, wsStatus)`

### app_build/frontend/src/components/AttackerTerminal.tsx — Real-Time Action Broker Telemetry Tape

**WHAT IT IS (IN SIMPLE ENGLISH):**

The terminal feed. Displays every single action executed by the agent with millisecond timestamps, latency, and status badges.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

Renders a monospace scrolling telemetry log styled in Charcoal (#111827) and Brand Green (#009B4D). Displays action ID, action type, target socket (IP:Port), latency in milliseconds, execution status (completed/denied), and expandable structured JSON output payloads.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`Exports: AttackerTerminal(actions, filterNode)`

### app_build/frontend/src/components/DetectionsMatrix.tsx — MITRE ATT&CK; Matrix & Threat Classifier View

**WHAT IT IS (IN SIMPLE ENGLISH):**

The threat analysis panel. Shows which MITRE techniques were triggered, the AI confidence level, and underlying packet evidence.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

Renders a real-time table of detected adversary behaviors. Displays MITRE technique IDs (e.g., T1046, T1110, T1071.004), technique names, confidence progress bars, source IP addresses, and expandable JSON evidence packets containing raw protocol details.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`Exports: DetectionsMatrix(detections)`

### app_build/frontend/src/components/HoneypotInspector.tsx — Honeypot Keystroke & Credential Forensics Drawer

**WHAT IT IS (IN SIMPLE ENGLISH):**

The forensic microscope. Shows attacker keystrokes, stolen password attempts, and HTTP headers with credentials safely redacted.

**HOW IT WORKS UNDER THE HOOD (TECHNICAL DEEP DIVE):**

Specialized forensic inspector displaying captured interaction sessions from Cowrie SSH, Web lures, DNS taps, and SMTP decoys. Features terminal replay view with confirmed '[REDACTED]' badges on sensitive credentials, timing intervals, and remote client banners.

**EXACT CODEBASE LOCATION (FILE PATH & KEY FUNCTIONS):**

`Exports: HoneypotInspector(sessions)`

# CHAPTER 5: DATA CONTRACTS & DATABASE SCHEMAS EXPLAINED

*In a high-security cyber-range, strict data contracts prevent injection attacks and guarantee non-repudiation. Every event emitted or stored in NetMirage adheres to formal Pydantic v2 schemas and SQLAlchemy ORM models.*

## 5.1 UNIVERSAL EVENT ENVELOPE JSON SCHEMA

```
{
  "run_id": "run_20261021_114500_a1b2",
  "event_id": "evt_9c8f3e1a",
  "timestamp": "2026-10-21T06:15:00.123456Z",
  "kind": "action",
  "source": "action-broker",
  "payload": {
    "action_id": "act_f4e2a109",
    "action_type": "attempt_ssh_to_lure",
    "target_ip": "10.0.20.10",
    "target_port": 22,
    "parameters": {
      "username": "root",
      "password": "[REDACTED]",
      "command": "cat /etc/passwd"
    },
    "status": "completed",
    "latency_ms": 24,
    "output": {
      "banner": "SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.6",
      "auth_status": "accepted_by_lure",
      "session_output": "uid=0(root) gid=0(root) groups=0(root)"
    }
  }
}
```

| Key | Type | Technical Purpose & Constraints |
|---|---|---|
| **run_id** | String(64) | Unique run instance identifier binding this event to its specific emulation lifecycle. |
| **event_id** | String(64) | Globally unique event token prefixed with 'evt_' for distributed event tracing and deduplication. |
| **timestamp** | ISO 8601 UTC | Microsecond-accurate Zulu timestamp ensuring strict chronological forensic sequencing. |
| **kind** | Enum String | Event domain category: 'system', 'action', 'observation', 'detection', 'policy', or 'honeypot'. |
| **source** | String(64) | Subsystem that originated the event: 'action-broker', 'safety-policy', 'protocol-analyzer', 'deception-router', or 'system'. |
| **payload** | JSON Object | Domain-specific structured data conforming to typed schemas with all sensitive credentials redacted. |

## 5.2 ACTION BROKER RESULT JSON SCHEMA

```
{
  "action": "discover_lab_services",
  "target": "10.0.10.42",
  "port": 80,
  "status": "completed",
  "latency_ms": 12,
  "output": {
    "scanned_target": "10.0.10.42",
    "open_services": [
      {
        "port": 80,
        "service": "HTTP/Reverse-Proxy",
        "banner": "Edge-Gateway-v2.1"
      }
    ],
    "message": "Service discovery completed on 10.0.10.42"
  }
}
```

| Key | Type | Technical Purpose & Constraints |
|---|---|---|
| **action** | String | The exact allowlisted action name executed from the 7 permitted functions. |
| **target** | IPv4 String | Validated destination IPv4 address inside the authorized /24 lab CIDR blocks. |
| **port** | Integer | Target port verified against the registered lab ports registry (e.g., 22, 53, 80). |
| **status** | Enum String | Terminal execution state: 'completed', 'denied', 'failed', or 'timed_out'. |
| **latency_ms** | Integer | Round-trip execution latency measured via high-precision hardware timers (perf_counter). |
| **output** | JSON Object / String | Structured result payload containing discovered banners, HTTP response previews, or session text. |

### 5.3 THE 6 IMMUTABLE DATABASE TABLES

| Table Name | Primary Key | Columns | Forensic Purpose |
|---|---|---|---|
| **runs** | id (String 64) | id, status, threat_score, active_profile, next_deception_zone, created_at, started_at, ended_at, metadata_json | Primary root table tracking the lifecycle, threat status, and deception state of each emulation experiment. |
| **agent_actions** | id (String 64) | id, run_id (FK), timestamp, action_type, target_ip, target_port, parameters, status, latency_ms, output | Immutable ledger of all safe action requests submitted by the planner and processed by the Action Broker. |
| **observations** | id (String 64) | id, run_id (FK), timestamp, source, source_ip, destination_ip, protocol, behavior_type, raw_payload, details | Passive network telemetry records derived from raw traffic analysis by the Protocol Analyzer. |
| **detections** | id (String 64) | id, run_id (FK), timestamp, source_ip, technique_id, technique_name, confidence, evidence | Adversary behavior classifications tagged with MITRE ATT&CK; technique IDs, names, and confidence scores. |
| **honeypot_sessions** | id (String 64) | id, run_id (FK), lure_type, session_id, source_ip, source_port, started_at, ended_at, commands_or_requests, metadata_json | Forensic records of decoy interactions (SSH, Web, DNS, SMTP) with credentials strictly sanitized. |
| **policy_decisions** | id (String 64) | id, run_id (FK), timestamp, threat_profile, confidence, decision, target_zone, evidence | Audit log of dynamic deception routing calculations and active zone containment decisions. |

# CHAPTER 6: HOW TO RUN, DEMO & TEST THE PROJECT

*This chapter provides the exact operational commands, faculty demonstration script, and test execution procedures required to run and defend NetMirage during the laboratory evaluation.*

## 6.1 PLATFORM STARTUP COMMANDS

**Single-Command PowerShell Launcher:**

```
powershell -ExecutionPolicy Bypass -File "<WORKSPACE_ROOT>/app_build/start_netmirage.ps1"
```

| Startup Step | Exact Command Execution |
|---|---|
| **Step 1: Virtual Environment** | `cd app_build/backend && python -m venv .venv && .venv/Scripts/pip install -r requirements.txt` |
| **Step 2: Database Setup** | `cd app_build/backend && .venv/Scripts/python -c "from app_build.backend.db import init_db; init_db()"` |
| **Step 3: Docker Lab Start** | `docker compose -f app_build/docker/docker-compose.lab.yml up -d (or rely on built-in emulator)` |
| **Step 4: Launch Backend API** | `cd && app_build/backend/.venv/Scripts/python -m uvicorn app_build.backend.main:app --host 0.0.0.0 --port 8000` |
| **Step 5: Launch Frontend UI** | `cd app_build/frontend && npm install && npm run dev -- --host 0.0.0.0 --port 5173` |

## 6.2 FACULTY DEMONSTRATION PLAYBOOK

| Time | Demonstration Action | What to Say & Point Out on Screen |
|---|---|---|
| **00:00 - 00:30** | **Project Overview & Topology** | Open http://localhost:5173. Point out the Ivory/Charcoal palette and the 5 VLAN subnet cards. Explain the zero-egress architecture. |
| **00:30 - 01:15** | **Initiate Run & Live WebSocket** | Click 'START RUN'. Point out the WebSocket indicator turning green, the threat score meter starting at 0, and the initial edge scan. |
| **01:15 - 02:00** | **Attacker Terminal & Action Broker** | Show the live Action Broker telemetry tape. Point out the millisecond latency, structured JSON output, and zero raw bash shell. |
| **02:00 - 02:45** | **Protocol Analysis & MITRE Matrix** | Scroll to the Detections Matrix. Point out MITRE Technique T1046, T1110, and T1071.004 appearing live with 0.85-0.95 confidence scores. |
| **02:45 - 03:30** | **Dynamic Deception & Route Diversion** | Show the Network Topology map. Point out the animated path dynamically re-routing from Edge to Cowrie SSH lure and Services lure in Tangerine Yellow. |
| **03:30 - 04:00** | **Honeypot Inspector & Credential Redaction** | Open the Honeypot Inspector drawer. Show the captured SSH login attempt and demonstrate that the password is safe [REDACTED]. |
| **04:00 - 04:30** | **Global Kill Switch Demonstration** | Click 'PAUSE' to freeze emulation; click 'RESUME' to continue; click 'TERMINATE' to trigger immediate container teardown. |

## 6.3 AUTOMATED TEST SUITE (24 VERIFIED TESTS)

```
& "<WORKSPACE_ROOT>/app_build/backend/.venv/Scripts/python.exe" -m pytest "<WORKSPACE_ROOT>/app_build/backend/tests" -v
```

| Test Module | Tests | Verified Subsystems & Safety Boundaries |
|---|---|---|
| **test_api_and_db.py** | **6 tests** | CRUD operations on all 6 tables, /api/health, /api/topology, run lifecycle, session redaction, WebSocket streaming. |
| **test_docker_policy.py** | **3 tests** | docker-compose validation, zero-egress internal: true check, container security boundaries (cap_drop: ALL, read_only). |
| **test_intel_and_deception.py** | **5 tests** | Credential redaction engine, Cowrie SSH normalization, Web lure normalization, MITRE mapper, Deception routing. |
| **test_safety_and_broker.py** | **9 tests** | Out-of-CIDR denied, unregistered ports denied, unapproved actions denied, rate limiting, kill switch, Action Broker execution. |
| **test_milestone1_e2e.py** | **1 test** | Full end-to-end multi-stage simulation validating all 10 lifecycle stages in a single automated integration run. |

# CHAPTER 7: SECURITY INVARIANTS, FAILURE MODES & VIVA DEFENSE

*This chapter consolidates NetMirage's core defense-in-depth engineering principles, fault tolerance mechanisms, and individual defense preparation topics for the four student candidates during faculty cross-questioning.*

## 7.1 NON-NEGOTIABLE SECURITY INVARIANTS

• **1. Zero Arbitrary Shell Execution**: Subprocess execution with shell=True is completely banned. All model decisions are translated through the 7 strictly parameterized Action Broker functions.

• **2. Fail-Closed Policy Engine**: Any missing parameter, unmapped port, out-of-CIDR IP address, or rate-limit violation immediately returns status: 'denied'. No operation executes without explicit affirmative policy clearance.

• **3. Strict Zero Egress**: Docker Compose bridges specify 'internal: true'. Default routing to the host's physical network adapters is omitted, preventing WAN data transmission.

• **4. Container Least Privilege**: All lab containers declare 'cap_drop: ALL', 'security_opt: [no-new-privileges:true]', 'read_only: true', and memory/CPU limits.

• **5. Universal Credential Redaction**: Passwords, secrets, tokens, and authorization headers are sanitized to '[REDACTED]' before persisting to SQLite or emitting over WebSockets.

## 7.2 SYSTEM FAULT TOLERANCE & FAILURE MODES

• **Docker Daemon Inactive**: The system automatically falls back to the deterministic zero-egress lab emulator, allowing 100% functional UI demonstration and testing without Docker installed.

• **Rate Limit Saturation**: If an autonomous agent exceeds 60 actions/minute, the Safety Policy Engine denies requests with HTTP 429 semantics, preserving host stability.

• **WebSocket Client Drop**: If the browser disconnects or reloads, the Event Bus retains event history in memory and replays all historical events upon reconnection.

• **Uncaught Planner Exception**: The LabRunner event loop wraps execution in a try-except block, automatically updating run status to 'failed' and cleanly releasing concurrency locks.

## 7.3 CANDIDATE DEFENSE MATRIX (EXAMINER CROSS-QUESTIONING)

| Candidate Name | Subsystem Focus | Core Files to Open | Technical Defense Point for Examiner |
|---|---|---|---|
| **Abhisumat Panda** (25BYB0202) | Network Virtualization, Subnetting Math & Docker Hardening | `app_build/docker/docker-compose.lab.yml & app_build/backend/tests/test_docker_policy.py` | Explains how the 5 /24 CIDRs are segregated, how Docker 'internal: true' modifies host routing tables, and why 'cap_drop: ALL' stops container breakouts. |
| **Atharva Parmar** (25BYB0180) | AI Planner, Safety Policy Engine & Action Broker | `app_build/backend/core/planner.py, safety_policy.py, action_broker.py` | Defends the 3-layer adversary loop, explains the 7-stage deterministic progression, and proves why the fail-closed policy engine rejects out-of-bounds IPs. |
| **Manas Patel** (25BYB0190) | Protocol Telemetry, MITRE ATT&CK; Mapping & Honeypot Forensics | `app_build/backend/intel/analyzer.py, mitre_mapper.py, honeypots.py, deception_policy.py` | Explains how TCP SYN scans and DNS queries are detected passively, how confidence scores (0.80-0.95) are calculated, and how regex credential redaction works. |
| **Varun Agarwal** (25BYB0186) | Control API, WebSocket Event Bus, Database & Operator Console | `app_build/backend/main.py, routes.py, event_bus.py, models.py, app_build/frontend/src/App.tsx` | Walks through the FastAPI REST routes, explains how the Event Bus manages asyncio queues and replay, and demonstrates real-time React state synchronization. |